Verification and Validation

Minsoo Ryu

Hanyang University

Topics Covered

1. Verification and Validation
2. Software Inspections
3. Static Analysis and Formal Methods

Verification vs. Validation (Boehm)

- **Verification**
  - "Are we building the product right"
  - The software should conform to its specification
- **Validation**
  - "Are we building the right product".
  - The software should do what the user really requires
  - Validation is a more general process and goes beyond the verification
- **Verification and validation should establish confidence that the software is fit for purpose**
  - This does NOT mean completely free of defects
  - Rather, it must be good enough for its intended use and the type of use will determine the degree of confidence that is needed

Two Complementary Approaches

- **Software inspections (static verification)**
  - Analyze and check work products such as the requirements document, design diagrams, and the program source code
  - Concerned with analysis of the static system representation to discover problems

- **Software testing (dynamic verification)**
  - Run an implementation of the software with test data
  - Concerned with exercising and observing product behaviour

Topics Covered

1. Verification and Validation
2. Software Inspections
3. Static Analysis and Formal Methods

Software inspections

- Software inspections generally focus on source code
  - The purpose is to find errors, omissions, and anomalies

- Three major advantages over testing
  - During testing, errors can mask (hide) other errors
  - Incomplete versions of a system can be inspected without additional cost
  - As well as searching for program defects, an inspection can also consider broader quality attributes of a program such as compliance with standards, portability, and maintainability

Success of Inspections

- Fagan (1986) reported that more than 60% of the errors in a program can be detected using informal program inspections
- Mills et al. (1987) suggest that a more formal approach to inspection can detect more than 90% of the errors in a program
- Selby and Basili (1987) empirically compared the effectiveness of inspections and testing
  - They found that static code reviewing was more effective and less expensive than defect testing in discovering program faults

Failure of Inspections

- It has proven to be difficult to introduce formal inspections into many software development organizations
- Software engineers with program testing experience are sometimes reluctant to accept that inspections can be more effective for defect detection than testing
- Managers may be suspicious because inspections require additional costs during design and development
  - It is difficult to convince a hard-pressed manager that this time can be made up later because less time will be spent on program debugging

The Inspection Process

- The notion of a formalized inspection process was first developed at IBM in the 1970s (Fagan)
  - Four roles: author, reader, tester, and moderator
  - The reader reads the code aloud to the inspection team, the tester inspects the code from the testing perspective, and the moderator organizes the process

- A number of alternative approaches (Gilb and Graham, 1993) have been developed
  - These are all based on a team with members from different backgrounds making a careful, line-by-line review of the program source code

Roles for Inspection

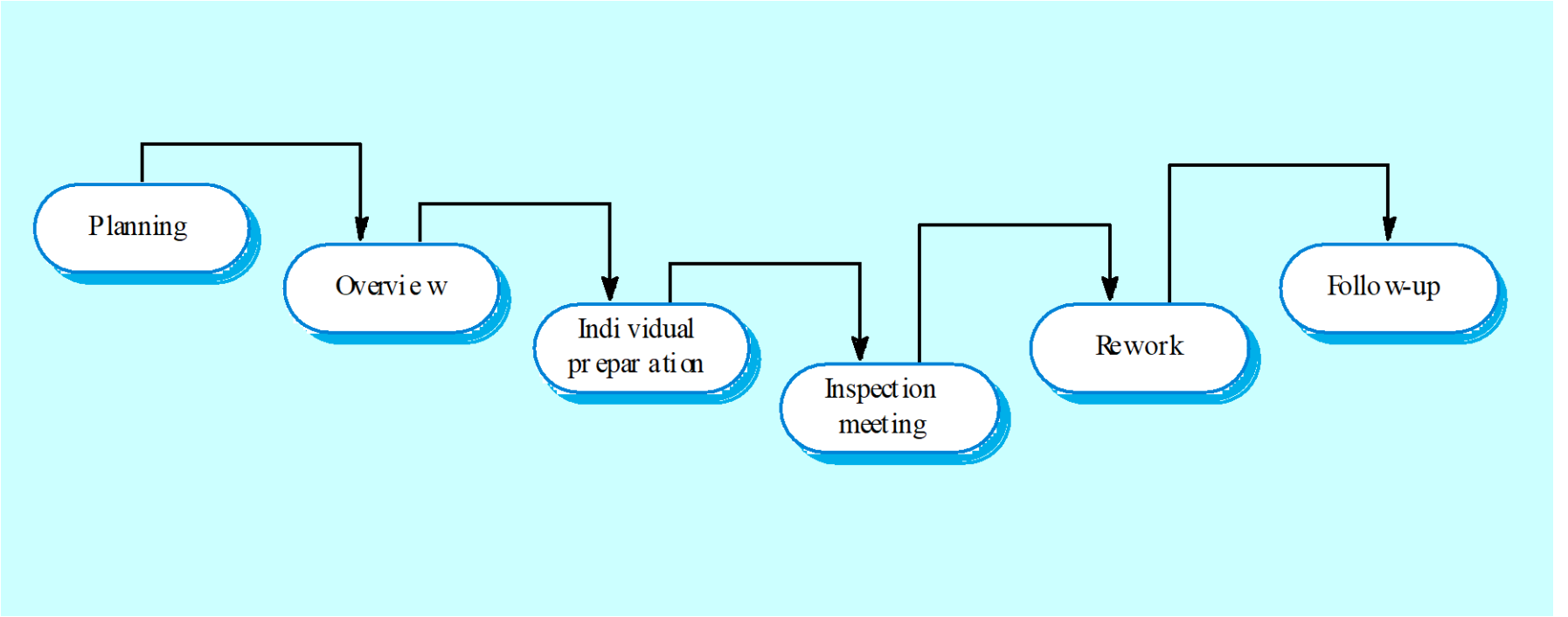
- Grady and Van Slack (1994) suggested six roles
  - The same person can take more than one role

Author or owner	The programmer or designer responsible for producing the program or document. Responsible for fixing defects discovered during the inspection process.
Inspector	Finds errors, omissions and inconsistencies in programs and documents. May also identify broader issues that are outside the scope of the inspection team.
Reader	Presents the code or document at an inspection meeting.
Scribe	Records the results of the inspection meeting.
Chairman or moderator	Manages the process and facilitates the inspection. Reports process results to the Chief moderator.
Chief moderator	Responsible for inspection process improvements, checklist updating, standards development etc.

Inspection Pre-conditions

- A precise specification must be available
- Team members must be familiar with the organisation standards
- Syntactically correct (compilable) code or other system representations must be available

The Inspection Process (by Michael Fagan in the mid-1970s)



The Inspection Process

- **Planning**
  - The moderator selects an inspection team, organizing a meeting room, and ensure that the material to be inspected and its specifications are complete
- **Overview**
  - The program to be inspected is presented to the inspection team
  - The author describes what the program is intended to do
- **Individual preparation**
  - Each inspection team member studies the specification and the program and looks for defects in the code

The Inspection Process

- **Inspection meeting**
  - Focus on defect detection, standards conformance, and poor quality programming
  - Suggest how these defects should be corrected
  - This meeting should be fairly short (no more than two hours)

- **Rework**
  - The program's author should make changes to correct the identified problems

- **Follow-up**
  - The moderator should decide whether a reinspection is required and that defects have been successfully fixed
  - The program is then approved by the moderator for release

Inspection Checklists

- During an inspection, a checklist of common programmer errors is often used to focus the discussion
  - The checklist varies according to programming language
  - Glib and Graham emphasize that each organization should develop its own inspection checklist based on local standards and practices

Inspection Check List

Data faults	Are all program variables initialised before their values are used? Have all constants been named? Should the upper bound of arrays be equal to the size of the array or Size -1? If character strings are used, is a de limiter explicitly assigned? Is there any possibility of buffer overflow?
Control faults	For each conditional statement, is the condition correct? Is each loop certain to terminate? Are compound statements correctly bracketed? In case statements, are all possible cases accounted for? If a break is required after each case in case statements, has it been included?
Input/output faults	Are all input variables used? Are all output variables assigned a value before they are output? Can unexpected inputs cause corruption?

Inspection Check List

Interface faults	Do all function and method calls have the correct number of parameters? Do formal and actual parameter types match? Are the parameters in the right order? If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	If a linked structure is modified, have all links been correctly reassigned? If dynamic storage is used, has space been allocated correctly? Is space explicitly de-allocated after it is no longer required?
Exception management faults	Have all possible error conditions been taken into account?

Inspection Rates

- **Inspection time**
  - 500 statements/hour during overview
  - 125 source statement/hour during individual preparation
  - 90-125 statements/hour can be inspected

- **Inspection is therefore an expensive process**
  - The cost of inspecting 100 lines of code is roughly equivalent to one person-day effort
  - However, the effort required for the program inspection is probably less than half the effort that would be required for equivalent defect testing

Lightweight Code Review

- Over-the-Shoulder: The reviewer physically looks over the author's shoulder and is walked through the code
  - Pros: Easy to implement, no tools required
  - Cons: Interrupts reviewers, hard to know which code is reviewed, impossible to measure

- Email Pass-Around: Code differences are emailed automatically by the version control system. Reviewers respond to changes when they have time
  - Pros: Easy to set up, nothing to purchase, works remotely
  - Cons: No tracking -- don't know if found defects were fixed, often reviews never happen because everyone assumes someone else looked at it, no metrics; reviews "fizzle out“ instead of finishing. Hard to follow conversations as emails get replied to and forwarded among several people

Lightweight Code Review

- **Tool-Based Review: Use a development tool specifically designed to assist in code review (A variety of open-source and commercial tools exist)**
  - **Pros: Removes most of the busywork associated with file-collection, discussion/defect management, and metrics/reporting. Tracks reviews and defects.**
  - **Cons: Have to learn and integrate another tool, can cost money.**

- **Pair-Programming: Two developers at the same keyboard and monitor working on the code together**
  - **Pros: Deep level of review; teams know review is happening but it's difficult to track.**
  - **Cons: Large time investment, some developers dislike it. Requires coordinating with someone else's schedule.**

Social Aspects of Code Review

- **The "Ego Effect"**
  - **Peer review invokes the ego**
    - "Jerry always forgets to check for NULL."
  - **Immediate improvement in code quality**
  - **Senior developers stop being lazy**

- **Personal Growth**
  - **if ("integrate".equals(s))**
  - **Surprising Effects**
    - Learning not related to the goal of the review
    - Reviewer learns too

- **Negative Emotions**
  - **The "Big Brother" Effect**
  - **Hurt feelings**



Topics Covered

1. Verification and Validation
2. Software Inspections
3. Static Analysis and Formal Methods

Automated Static Analysis

- Static analyzers are software tools for source text processing
  - They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the V & V team
  - They are very effective as an aid to inspections - they are a supplement to but not a replacement for inspections

- "Too many false positives" is probably the most common excuses for avoiding static analysis
  - A "false positive" occurs when a static analysis tool incorrectly reports that a static analysis rule was violated

Static Analysis Checks

Fault class	Static analysis check
Data faults	Variables used before initialisation Variables declared but never used Variables assigned twice but never used between assignments Possible array bound violations Undeclared variables
Control faults	Unreachable code Unconditional branches into loops
Input/output faults	Variables output twice with no intervening assignment
Interface faults	Parameter type mismatches Parameter number mismatches Non-usage of the results of functions Uncalled functions and procedures
Storage management faults	Unassigned pointers Pointer arithmetic

Static Analyzers for C

- **Static analyzers are particularly valuable when C is used**
  - **C does not have strict type rules and error-prone features such as pointers**
    - C, C++, and Java are weakly typed language since they provide a number of ways to get around the type system such as casts and implicit conversion
  - **But Java has removed some error-prone features**
    - All variables must be initialized
    - No goto statements, so unreachable code is less likely to be created
    - Storage management is automatic

Static Analyzers for C

- Lint
  - Lint first appeared (outside of Bell Labs) in the seventh version (V7) of the UNIX operating system in 1979
- Coverity (originated from Stanford Checker)
  - http://www.coverity.com
- CCured (UC Berkeley)
  - http://manju.cs.berkeley.edu/ccured/
- Introspector (GCC)
- Others
  - HP Code Advisor
  - MS Visual Studio (VS2005)

Lint Static Analysis

```
138% more lint_ex.c
#include <stdio.h>
printarray (Anarray)
 int Anarray;
{   printf("%d?Anarray);  }

main ()
{
 int Anarray[5]; int i; char c;
 printarray (Anarray, i, c);
 printarray (Anarray) ;
}

139% cc lint_ex.c
140% lint lint_ex.c

lint_ex.c(10): warning: c may be used before set
lint_ex.c(10): warning: i may be used before set
printarray: variable # of args. lint_ex.c(4) :: lint_ex.c(10)
printarray, arg. 1 used inconsistently lint_ex.c(4) :: lint_ex.c(10)
printarray, arg. 1 used inconsistently lint_ex.c(4) :: lint_ex.c(11)
printf returns value which is always ignored
```

Verification and Formal Methods

- Formal methods are based on mathematical representations of the software, usually a formal specification
  - Formal methods are mainly concerned with mathematical analysis of the specification
  - They transform the specification to a more detailed, semantically equivalent representation and/or formally verify that one representation of the system is semantically equivalent to another representation
- They are the ultimate static verification technique

Cost of Formal Methods

- Verifying a nontrivial software takes a great deal of time and requires specialized tools such as theorem provers and mathematical expertise
  - It is therefore an extremely expensive process and, as the system size increases, the costs of formal verification increase disproportionately
- Many people think that formal verification is not cost-effective
  - The same level of confidence can be achieved more cheaply by using other validation techniques such as inspection and system testing

Reliability and Formal Methods

- It is sometimes claimed that the use of formal methods for system development leads to more reliable and safer systems

- However, formal specification and proof do not guarantee that the software will be reliable in practical use
  - The specification may not reflect the real requirements of system users
  - The proof may contain errors
  - The proof may assume a usage pattern which is incorrect


Q & A